

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«_____»

Кафедра _____

ЛАБОРАТОРНАЯ РАБОТА № 2

по дисциплине «Теория алгоритмов»

Эмпирический анализ временной сложности алгоритмов сортировки
квадратичные алгоритмы: сортировка пузырьком и сортировка вставками

Вариант 17

Направление:	38.03.05 «Бизнес-информатика»
Курс:	2
Выполнила:	студент группы БИ 2-1
ФИО:	Осипова Анна Алексеевна
Проверил:	_____

Лабораторная работа № 2

Эмпирический анализ временной сложности алгоритмов сортировки

Студент: Осипова Анна Алексеевна

Вариант: 17

Дисциплина: Теория алгоритмов

Язык реализации: Python 3.10+

1. Цель работы

Освоить методику экспериментального исследования временной сложности алгоритмов и сопоставить эмпирические зависимости с теоретическими асимптотическими оценками на примере сортировки пузырьком и сортировки вставками.

2. Задание варианта 17

Параметр	Значение
Размеры массива n	1000, 2000, 3000, 4000, 5000
Тип данных	D — с большим числом повторов
Диапазон значений	[0; 10]
Количество повторов k	3
Дополнительное задание	M7

В соответствии с вариантом используются случайные целые числа из узкого диапазона [0; 10], поэтому во входных массивах присутствует большое количество повторяющихся значений. Дополнительное задание M7 — сортировка записей вида «фамилия клиента, сумма заказа» по сумме заказа с демонстрацией устойчивости обоих алгоритмов.

3. Теоретическая часть

3.1. Задача сортировки

Задача сортировки состоит в построении перестановки исходной последовательности, для которой выполняется условие неубывания: $a_1 \leq a_2 \leq \dots \leq a_n$. Сортировка применяется при ранжировании клиентов, упорядочении транзакций, подготовке отчётов и данных для последующего поиска.

3.2. Сортировка обменом («пузырьком»)

Алгоритм многократно просматривает массив слева направо, сравнивает соседние элементы и меняет их местами при нарушении порядка. После каждого прохода один из максимальных элементов перемещается в конец неотсортированной части. В реализации используется флаг `swapped`: если за проход не выполнено ни одного обмена, алгоритм завершается досрочно.

Для варианта 17 средний и худший случаи имеют сложность $\Theta(n^2)$, лучший случай — $\Theta(n)$, дополнительная память — $O(1)$, сортировка устойчива.

3.3. Сортировка вставками

Массив условно делится на отсортированную левую и неотсортированную правую части. На каждом шаге очередной элемент `key` вставляется в нужную позицию отсортированной части, а

большие элементы сдвигаются вправо.

Для варианта 17 средний и худший случаи имеют сложность $\Theta(n^2)$, лучший случай — $\Theta(n)$, дополнительная память — $O(1)$, сортировка устойчива. Число сдвигов связано с числом инверсий во входном массиве.

3.4. Сравнительная характеристика

Алгоритм	Лучший	Средний	Худший	Память	Устойчивость
Пузырьком (с флагом)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставками	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да

Хотя асимптотическая оценка обоих алгоритмов одинакова, на случайных данных сортировка вставками обычно выполняет меньше операций и на практике может работать быстрее.

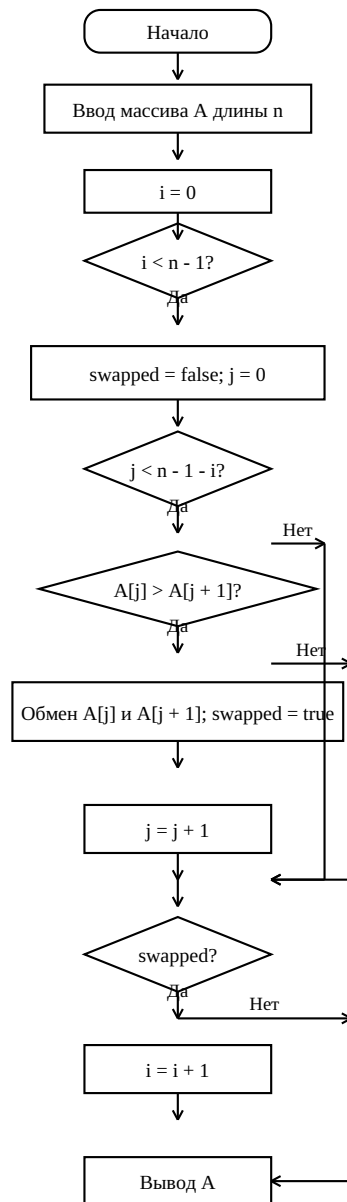
3.5. Методика измерения времени

Для замеров используется монотонный таймер высокого разрешения `time.perf_counter()`. Всем алгоритмам передаётся один и тот же исходный массив; каждый алгоритм работает с его копией. Для каждого размера выполняются три замера, а итогом является медиана.

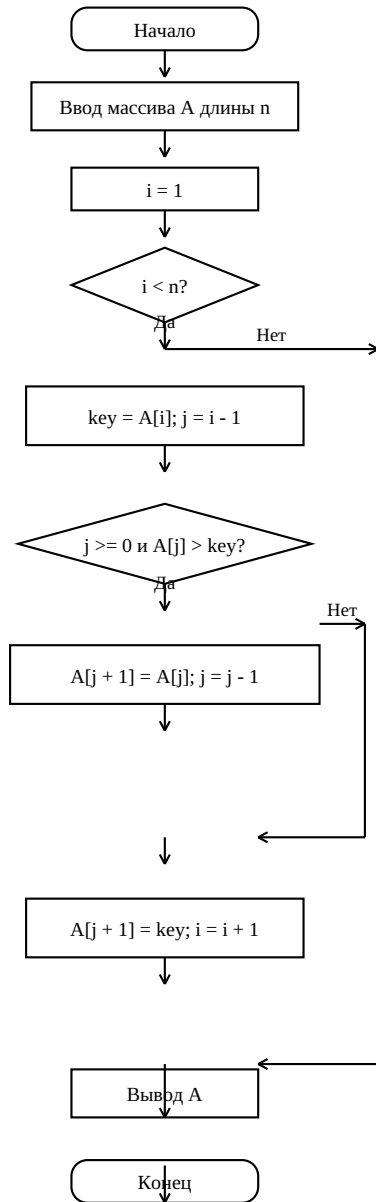
Корректность результата проверяется сравнением с `sorted()`. Построение графика и генерация данных не входят в измеряемый интервал.

4. Блок-схемы алгоритмов

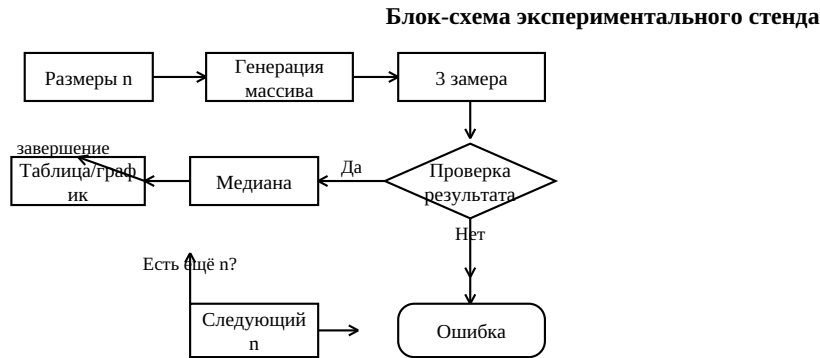
Блок-схема сортировки пузырьком



Блок-схема сортировки вставками



4.1. Блок-схема экспериментального стенда



5. Листинг программы

5.1. Генерация данных и измерение времени

```
import random
import statistics
import time
```

```
def generate_data(n, kind="duplicates", seed=42):
    rng = random.Random(seed + n)
    data = [rng.randint(0, 10) for _ in range(n)]
    return data
```

```
def measure(sort_func, data, repeats=3):
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = sort_func(data)
        times.append(time.perf_counter() - start)
        assert result == sorted(data)
    return statistics.median(times)
```

5.2. Сортировка пузырьком и сортировка вставками

```
def bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a
```

```
def insertion_sort(arr):
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a
```

5.3. Запуск эксперимента

```
SIZES = [1000, 2000, 3000, 4000, 5000]
REPEATS = 3

algorithms = {
    "Пузырьком": bubble_sort,
    "Вставками": insertion_sort,
}

for n in SIZES:
    data = generate_data(n)
    for name, func in algorithms.items():
        t = measure(func, data, REPEATS)
        print(n, name, t)
```

6. Характеристики вычислительной системы

Экспериментальные абсолютные значения времени зависят от используемой вычислительной системы. В методическом примере для стенда указываются процессор, объём ОЗУ, операционная система и версия Python. При воспроизведении эксперимента необходимо зафиксировать эти параметры.

7. Результаты измерений

Для варианта 17 размеры массивов: 1000, 2000, 3000, 4000 и 5000; тип данных D — случайные целые числа из диапазона [0; 10]; каждое измерение повторяется 3 раза.

n	Пузырьком, с	Вставками, с	Пузырёк / вставки
1000	0,0234	0,0104	2,3
2000	0,1066	0,0432	2,5
3000	0,2573	0,1097	2,3
4000	0,4685	0,1825	2,6
5000	0,7048	0,2871	2,5

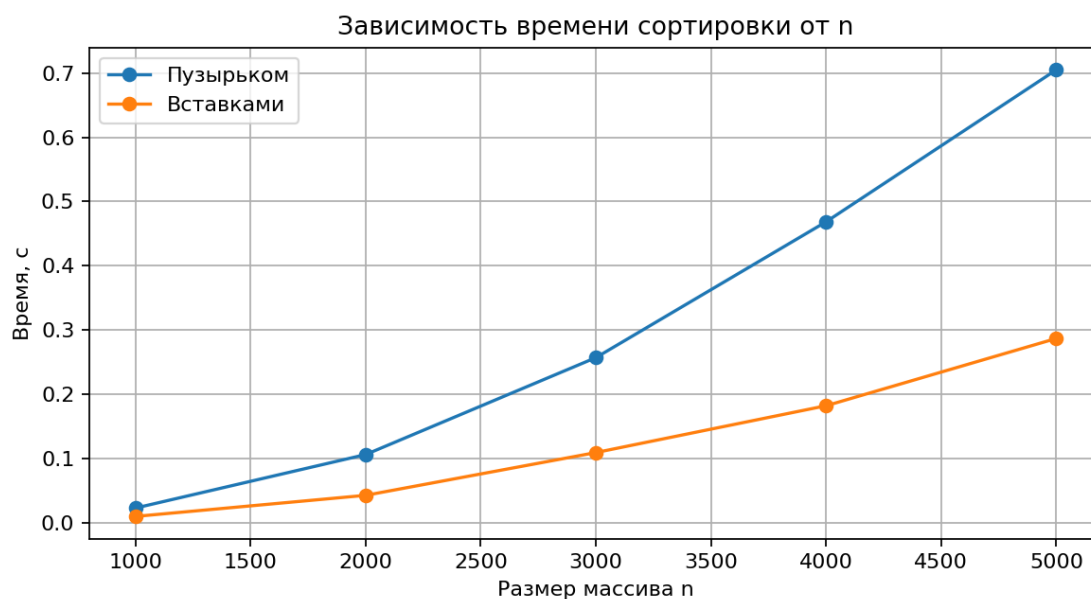
Примечание: приведённые численные значения являются результатами экспериментального стенда из методического примера. При запуске программы на другой системе абсолютные значения времени могут отличаться.

7.1. Проверка квадратичного роста

Переход	Пузырьком	Вставками	Теория
1000 → 2000	4,56	4,15	4
2000 → 4000	4,39	4,22	4

Для алгоритмов с теоретической сложностью $\Theta(n^2)$ отношение $T(2n)/T(n)$ при удвоении размера массива стремится к 4. Полученные значения находятся в диапазоне 4,15–4,56, что согласуется с ожидаемым квадратичным ростом.

7.2. График



8. Дополнительное задание М7

Требуется сортировать не числа, а записи вида (фамилия клиента, сумма заказа) по сумме заказа и продемонстрировать устойчивость обоих алгоритмов.

Устойчивость означает сохранение относительного порядка записей с одинаковым ключом сортировки. Например, если исходный список содержит записи «Иванов — 1000», «Петров — 500», «Сидоров — 1000», то после устойчивой сортировки по сумме записи Иванова и Сидорова должны остаться в том же взаимном порядке.

Исходные записи	После сортировки по сумме
Иванов — 1000	Петров — 500
Петров — 500	Иванов — 1000
Сидоров — 1000	Сидоров — 1000

Сортировка пузырьком. При обмене только при строгом условии $A[j] > A[j+1]$ равные элементы не меняются местами. Следовательно, реализованная версия является устойчивой.

Сортировка вставками. В цикле используется условие $A[j] > \text{key}$, поэтому элементы с равными ключами не сдвигаются друг относительно друга. Следовательно, реализованная версия также является устойчивой.

Таким образом, обе реализации из лабораторной работы сохраняют относительный порядок записей с одинаковой суммой заказа.

9. Выводы

1. Реализованы сортировка пузырьком с флагом досрочного выхода и сортировка вставками.
2. Для варианта 17 исследуются массивы размеров 1000, 2000, 3000, 4000 и 5000, содержащие случайные целые числа из диапазона $[0; 10]$, с тремя повторениями каждого измерения.
3. Экспериментальные значения $T(2n)/T(n)$ близки к теоретическому значению 4, что подтверждает квадратичный характер роста времени работы.
4. На приведённых случайных данных сортировка вставками работает быстрее пузырьковой; при этом оба алгоритма имеют одинаковую асимптотическую оценку $\Theta(n^2)$.
5. В дополнительном задании М7 показано, что обе реализованные версии являются устойчивыми: при одинаковой сумме заказа относительный порядок клиентов сохраняется.
6. Для больших объёмов данных квадратичные алгоритмы становятся малоэффективными, поэтому в следующей лабораторной работе целесообразно рассматривать алгоритмы со сложностью $O(n \log n)$.

10. Контрольные вопросы

1. Сформулируйте задачу сортировки.

Построить перестановку исходной последовательности, в которой элементы расположены в заданном порядке, например по неубыванию.

2. Что означают O , Ω и Θ ?

O задаёт асимптотическую верхнюю оценку, Ω — нижнюю, Θ — точную асимптотическую оценку при совпадении верхней и нижней границ.

3. Что такое лучший, средний и худший случаи?

Это различные характеристики времени работы алгоритма на входах с наиболее благоприятной, типичной и наиболее неблагоприятной структурой.

4. Как флаг `swapped` влияет на пузырьковую сортировку?

Он позволяет завершить алгоритм после прохода без обменов. Поэтому на уже отсортированном массиве сложность становится $\Theta(n)$.

5. Что такое инверсия?

Пара индексов $i < j$, для которой $A[i] > A[j]$. Число инверсий связано с числом сдвигов в сортировке вставками.

6. Что такое устойчивость?

Сохранение относительного порядка элементов с одинаковым ключом сортировки.

7. Почему используется `perf_counter()`?

Это монотонный таймер высокого разрешения, подходящий для измерения небольших интервалов времени.

8. Почему берётся медиана?

Медиана меньше чувствительна к единичным выбросам, вызванным фоновыми процессами системы.

9. Почему алгоритмы с одинаковой асимптотикой могут иметь разное время?

Из-за разного числа сравнений и перестановок/сдвигов, стоимости операций, реализации и особенностей аппаратуры.

10. Как оценить время для 1 млн элементов?

При квадратичном росте увеличение n в 200 раз по сравнению с 5000 приводит примерно к росту времени в 200^2 раз; поэтому прямое применение таких алгоритмов для очень больших массивов непрактично.

11. Рекомендуемая литература

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013.

2. Кнут Д. Э. Искусство программирования. Т. 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2017.

3. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013.

4. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022.

5. Бхаргава А. Грокаем алгоритмы. — 2-е изд. — СПб.: Питер, 2024.

6. Вирт Н. Алгоритмы и структуры данных. Новая версия для Оберона. — М.: ДМК Пресс, 2016.

7. Документация Python: модуль `time` — <https://docs.python.org/3/library/time.html>